

GraphOne / FrontierAtlas ? AI Intelligence Pipeline

Production Architecture & Scalability Design

1. Architecture Overview

Sources ? Async Crawlers ? Raw Storage ? Normalization ? LLM Orchestrator ? Entity Resolution ? Validation ? SQLite/PostgreSQL ? API/Google Sheets

The pipeline is designed as independently scalable stages. Crawling, enrichment, LLM extraction and entity resolution are separated so that each stage can be scaled horizontally without changing application logic. Raw source URLs and timestamps are retained for complete traceability and to prevent hallucinated records.

2. Scale Strategy ? 500K+ Records

Component	Production Strategy
Crawlers	asyncio + aiohttp/Playwright; bounded concurrency using semaphores.
Work Queue	Distributed queue such as Redis Streams/Kafka/SQS; partition by source/entity.
Workers	Stateless crawler, parser and LLM workers; horizontally scalable.
Deduplication	Deterministic URL/content hash before expensive LLM processing.
Persistence	Batch writes and connection pooling; PostgreSQL recommended for production.
Scaling	Increase worker count/infrastructure capacity; application logic remains unchanged.

3. LLM Orchestration ? 413 & 429 Handling

Fallback chain: Gemini Flash ? Groq Llama ? DeepSeek. Each provider is isolated behind a common interface so failures do not stop the pipeline.

413 / context overflow: input is cleaned and semantically chunked before inference. A configurable maximum character/token budget is enforced. Chunks use controlled overlap to preserve context. If a provider still reports a payload-size error, the chunk is reduced and retried rather than sending the same oversized payload again.

429 / rate limiting: retry with exponential backoff and random jitter. Respect Retry-After when supplied. Provider-specific concurrency limits prevent a single provider from being overloaded. After retry exhaustion, the next LLM provider is used.

4. Freshness & Exactly-Once Processing

News and job records are normalized to UTC ISO-8601 timestamps. Sources are checked for published/modified metadata, JSON-LD, OpenGraph/meta tags and visible date text. Relative dates such as '2 hours ago' are converted using the crawl timestamp. When no strict date exists, source-specific heuristics and last-seen state are used.

Every item receives a deterministic fingerprint based on canonical URL plus normalized content/title. A distributed uniqueness key such as **source + URL hash** prevents multiple crawler nodes from processing the same article/job. Database unique constraints provide a final idempotency barrier.

5. Research Paper & GitHub Enrichment

ArXiv/Papers with Code records are normalized into the RESEARCH_PAPER schema. Paper URLs are retained as source evidence. Associated GitHub repositories are resolved and current star counts are retrieved through GitHub APIs. Retry handling and rate-limit awareness protect GitHub API usage. The stored collectedAt timestamp makes the metric auditable.

6. Deterministic Entity Resolution

Extracted organization/product names are normalized using Unicode normalization, case-folding, punctuation removal, whitespace normalization and known aliases. A seed canonical-entity mapping is checked first. Exact normalized matches are preferred; safe alias/fuzzy matching is applied only when confidence is above a configured threshold.

Example: **OpenAI**, **OpenAI, Inc.** and **Open AI** resolve to canonical **OpenAI**. Every resolution is recorded in the Entity Mapping Log with raw name, canonical name and mapping method for auditability.

7. Anti-Bot & JavaScript-Rendered Sources

The crawler first uses lightweight asynchronous HTTP requests. For pages requiring JavaScript rendering, Playwright Async is used with bounded browser concurrency. The system respects robots.txt, source terms and rate limits and avoids attempting to circumvent authentication or access controls. Browser sessions are reused where safe, timeouts are enforced, and failed pages are placed into a retry/dead-letter queue.

For heavily protected high-value sources, the production design favors permitted APIs, RSS feeds, licensed datasets or publisher-provided feeds instead of relying on brittle captcha bypass techniques.

8. Storage Strategy

Layer	Technology	Purpose
Primary	PostgreSQL	Canonical entities, signals, jobs, papers, constraints and transactions.
Raw	Object storage	HTML/JSON/PDF snapshots for replay and audit.
Queue	Kafka / Redis Streams	Distributed work scheduling and backpressure.
Vector	pgvector	Semantic similarity and entity candidate retrieval.
Graph	Neo4j / graph layer	Startup-founder-product-paper-job relationships.
Current Trial	SQLite	Portable local development and demonstrated pipeline output.

9. Reliability & Observability

All stages use structured logging, explicit timeouts, retry counters and failure states. Metrics should include records discovered, accepted, rejected, duplicate rate, HTTP errors, LLM latency, token usage, 413/429 counts and provider fallback rate. Failed items are retrievable without restarting the complete acquisition job.

10. Data Quality & Traceability

LLMs are used only for structuring extracted evidence, not inventing missing facts. Each record retains a legitimate source URL and collection timestamp. Required fields are validated against the canonical schema before persistence. Invalid or unsupported records are rejected or flagged instead of silently creating synthetic data.

11. Current Demo Implementation

The submitted implementation demonstrates asynchronous acquisition, GitHub enrichment, LLM fallback/resilience, payload chunking, deterministic entity resolution, mapping logs, SQLite-backed datasets and FastAPI endpoints. The design separates the current local demo storage from the production horizontal-scaling architecture described above.

Design principle: preserve raw evidence, make every stage idempotent, bound concurrency, retry transient failures, and scale workers rather than rewriting code.